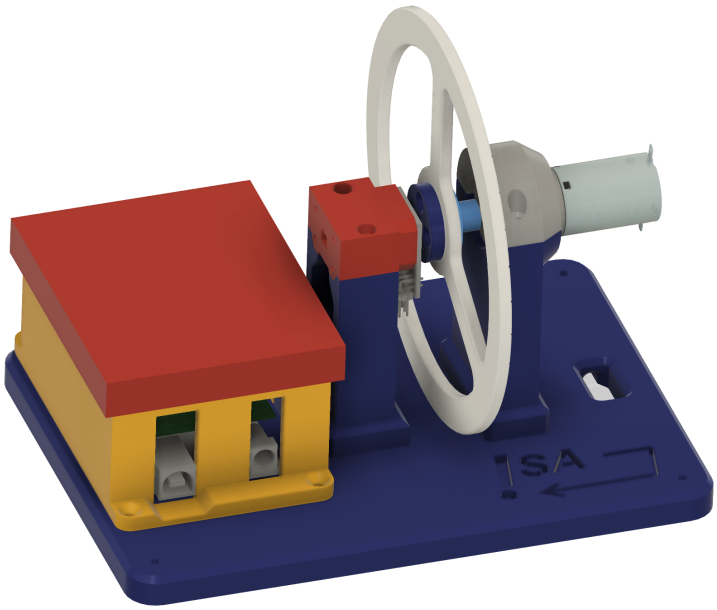
Manual de Usuario

Equipo de Prácticas de Control de Motor CC



Departamento de Ingeniería de Sistemas y Automática (ISA)

Universidad de Sevilla · ialvarado@us.es

November 7, 2026

En qué consiste este equipo: un motor DC Maxon con encoder magnético de 14 bits, controlado en lazo cerrado desde MATLAB/Simulink a través de un Arduino. El lazo de control *corre íntegramente en el PC*: Arduino solo lee el encoder y aplica el PWM que MATLAB le ordena cada **10 ms** (100 Hz). La comunicación es por USB/serie a 115 200 baudios.

Lo que necesita el profesor: conectar el USB, identificar el puerto serie en MATLAB, y lanzar el experimento. El resto es automático.

Contents

1 Descripción del equipo **3**
1.1 Componentes principales . . . 3
1.2 Esquema de conexiones . . . 3
1.3 Parámetros del sistema . . . 4

2 Puesta en marcha inicial (una sola vez) **4**
2.1 Instalar las librerías de Arduino . . . 4
2.2 Cargar el firmware en el Arduino . . . 4
2.3 Comprobar el cableado de la placa shield . . . 5

3 Uso en cada sesión de prácticas **5**
3.1 Paso 1 — Conectar el hardware . . . 5
3.2 Paso 2 — Seleccionar el puerto serie desde Simulink . . . 6
3.3 Paso 3 — Lanzar el experimento desde Simulink . . . 6
3.4 Paso 4 — Guardar los datos del experimento . . . 7

4 Referencia rápida **7**
4.1 Protocolo de comunicación serie . . . 7
4.2 Pines del Arduino / Shield . . . 8
4.3 Conversión señal de control → tensión media . . . 8

5 Resolución de problemas frecuentes **8**

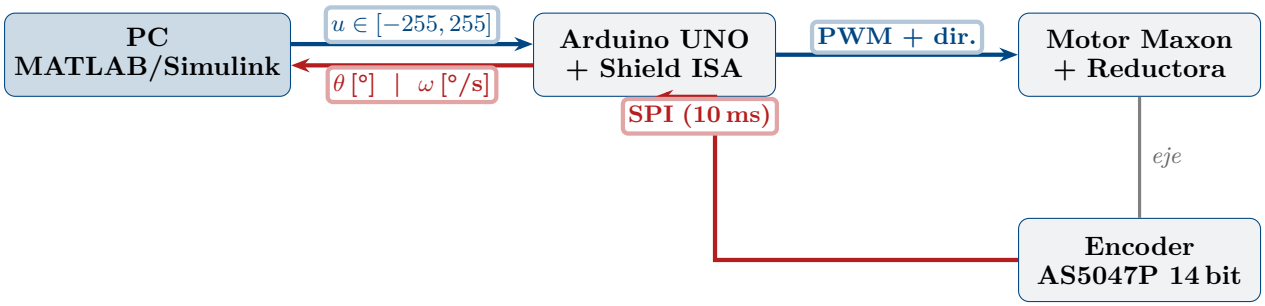
6 Repositorio y recursos **9**

1 Descripción del equipo

1.1 Componentes principales

Componente	Modelo	Función
Motor	Maxon A-max 22	Motor DC de precisión con reductora planetaria GP 22 B
Encoder	ams AS5047P	Encoder magnético 14 bits (16 384 pasos/vuelta), montado en el lado de carga
Driver	TB6612FNG	Driver de motor dual integrado en la placa shield
Microcontrolador	Arduino UNO	Lee el encoder, aplica PWM y se comunica con MATLAB por USB
Placa shield	ISA custom	Apilable sobre el Arduino; integra driver + conector encoder

1.2 Esquema de conexiones



⟶ Señal de control ⟶ Medidas del sistema — Acoplamiento mecánico

Nota

El lazo de control completo reside en MATLAB. Arduino actúa como **interfaz hardware en tiempo real**: marca la cadencia del experimento mediante interrupciones del Timer2 cada **10 ms** (100 Hz), lee el encoder por SPI, calcula la velocidad numérica ($\Delta\theta/T_s$) y aplica al driver la señal de control recibida de MATLAB.

Acumulación de ángulo: el encoder AS5047P entrega el ángulo en el rango $[0°, 360°)$ por vuelta. El firmware cuenta las vueltas completas y acumula el ángulo, de modo que la salida es un ángulo irrestricto $\in (-\infty, +\infty)$ que crece (o decrece) continuamente, sin saltos ni reinicios, igual que un encoder incremental.

Observador de Luenberger (desactivado por defecto): estima posición y velocidad filtradas en lugar de la derivada cruda. Se activa descomentando cuatro líneas al final del bloque de envío en `main.ino`.

1.3 Parámetros del sistema

Parámetro	Valor	Notas
Periodo de muestreo T_s	10 ms (100 Hz)	Fijado en el firmware de Arduino
Velocidad de comunicación	115 200 baudios	USB/serie
Rango señal de control	$u \in [-255, 255]$	Equivale a $\pm V_{\text{alim}}$ (ver nota)
Resolución encoder	$\approx 0.022°$/paso	16 384 pasos/vuelta
Modelo aproximado	$G(s) = K/(\tau s + 1)$	K y τ dependen de V_{alim}

△ Tensión de alimentación

La señal $u \in [-255, 255]$ se mapea a $\pm V_{\text{alim}}$, que depende de cómo se alimente el equipo:

- **Alimentación externa (recomendado):** máx. **9 V**; en la práctica se recomienda **8,4 V** para evitar calentamiento excesivo del motor. $u = 255 \Rightarrow 8{,}4$ V.
- **Solo USB (sin fuente externa):** el motor funciona a $\approx \pm 5$ V. Rango de velocidades reducido, pero perfectamente funcional para pruebas y para las sesiones en las que no se dispone de fuente de alimentación.

El modelo $G(s) = K/(\tau s + 1)$ debe identificarse experimentalmente para cada tensión de alimentación utilizada.

2 Puesta en marcha inicial (una sola vez)

Esta sección solo es necesaria la **primera vez** que se configura el equipo, o si se cambia el Arduino. No hace falta repetirla en cada sesión de prácticas.

2.1 Instalar las librerías de Arduino

1. Abrir **Arduino IDE 2.x**.
2. Ir a `Herramientas → Gestor de librerías`.
3. Buscar e instalar `AS5047P` (autor: Marginally Clever Robots).
4. Buscar e instalar `MsTimer2` (autor: Javier Valencia).

■ Nota

Estas librerías solo se instalan una vez en el PC del laboratorio. Si se cambia de ordenador, hay que repetir este paso.

2.2 Cargar el firmware en el Arduino

1. Conectar el Arduino al PC por USB.
2. Abrir el archivo `firmware/arduino/main/main.ino` del repositorio.
3. Seleccionar la placa correcta: `Herramientas → Placa → Arduino UNO`.
4. Seleccionar el puerto: `Herramientas → Puerto → COMx` (Windows) o `/dev/tty.usbmodem...` (Mac/Linux).

5. Pulsar el botón ↑ **Subir** (o `Ctrl+U`).
6. Esperar a que aparezca el mensaje *"Subida exitosa"*.

✔ Correcto

Para verificar que el firmware funciona: abrir el **Monitor Serie** del IDE (`Ctrl+Shift+M`), configurarlo a `115200 baudios`. Aparecerán **caracteres extraños o símbolos sin sentido** actualizándose a gran velocidad — esto es completamente normal. Los datos viajan codificados en binario, no como texto legible, de modo que el Monitor Serie los muestra como galimatías. Lo importante es que *aparezca algo*: si la pantalla permanece en blanco, revisar el cableado del encoder.

△ Atención

El firmware ya está cargado en el equipo del laboratorio. Solo hay que volver a cargarlo si se reemplaza el Arduino o se actualiza el firmware.

2.3 Comprobar el cableado de la placa shield

La placa shield se apila directamente sobre el Arduino (conexión sin cables). Verificar que:

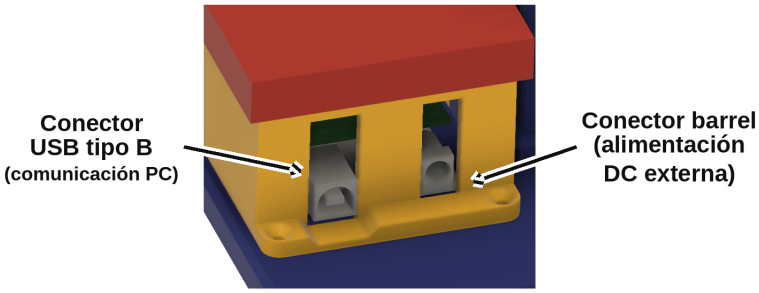
- La shield está bien asentada sobre el Arduino (todos los pines encajados).
- El cable del motor está conectado a los bornes del driver.
- El cable del encoder (conector JST de 7 pines) está conectado a la shield.
- La alimentación está conectada al conector de alimentación del propio Arduino (recomendado: **8,4 V**; máx. 9 V). Si no se dispone de fuente externa, el equipo funciona únicamente con el USB a ≈5 V.

3 Uso en cada sesión de prácticas

Este es el procedimiento habitual al inicio de cada sesión. Con el equipo ya configurado, tarda menos de dos minutos.

3.1 Paso 1 — Conectar el hardware

1. Conectar el cable USB del Arduino al PC.
2. **Opcional:** conectar la fuente de alimentación al conector barrel del Arduino (recomendado **8,4 V**, máx. 9 V). Sin fuente externa el equipo funciona con el USB solo (≈5 V), con menor rango de velocidades.



Detalle del frontal del equipo. Izquierda: conector USB tipo B (comunicación con el PC). Derecha: ***conector barrel*** *(toma de alimentación DC externa, 8,4 V recomendados).*

3. El LED del Arduino debe encenderse. ✔

△ Atención

Conectar siempre primero el USB y *después* la alimentación externa. Al desconectar, en orden inverso: primero la alimentación, luego el USB.

3.2 Paso 2 — Seleccionar el puerto serie desde Simulink

El puerto se configura directamente en el modelo Simulink, sin necesidad de comandos adicionales en MATLAB.

1. Abrir el modelo `software/matlab/MotorCC_V3.slx`.
2. Hacer **doble clic** sobre el bloque central **"Motor CC"**. Se abrirá una ventana con los parámetros del bloque, donde aparece una lista desplegable con los puertos serie disponibles en el PC.
3. Seleccionar el puerto que corresponde al Arduino y pulsar **OK**.

■ ¿Cuál es el puerto del Arduino?

Si no es evidente qué puerto es el Arduino:

1. **Desconectar** el cable USB del Arduino.
2. Hacer doble clic sobre el bloque **"Motor CC"** y anotar los puertos que aparecen en la lista.
3. **Reconectar** el USB del Arduino.
4. Volver a hacer doble clic sobre el bloque: el puerto nuevo que aparece ahora es el del Arduino. Seleccionarlo y pulsar OK.

3.3 Paso 3 — Lanzar el experimento desde Simulink

1. Con el puerto ya seleccionado en el bloque **"Motor CC"** (Paso 2), pulsar el botón **Run** (▶) en la barra de herramientas de Simulink.
2. El modelo se conecta automáticamente al Arduino y comienza a ejecutar el lazo de control a 100 Hz.

3. Para detener el experimento, pulsar el botón **Stop**.

△ Parar la simulación con el motor en reposo

Detener la simulación (botón **Stop**) *con el motor en movimiento* puede ser peligroso: al perderse la comunicación, el driver mantiene indefinidamente el **último valor de PWM** recibido y el motor continúa girando sin control. Esperar siempre a que el motor esté **parado** (señal de control $u = 0$) antes de pulsar Stop, o bien reducir u a cero desde Simulink antes de detener.

✔ Correcto

Si todo va bien, en el scope de Simulink se verán en tiempo real la posición angular (en grados) y la señal de control aplicada. El motor responderá a la referencia configurada.

3.4 Paso 4 — Guardar los datos del experimento

El modelo `MotorCC_V3.slx` no incluye bloques de guardado de datos: se espera que cada alumno añada en su propio modelo los bloques necesarios (*To Workspace*, *To File*, *Scope* con autoguardado, etc.) según las instrucciones de cada práctica.

■ Nota

Las variables disponibles en cada ciclo de Simulink son la posición angular (°), la velocidad angular (°/s) y la señal de control $u \in [-255, 255]$. Consultar el guión de cada práctica para saber exactamente qué bloques añadir y qué nombre dar a las variables exportadas.

4 Referencia rápida

4.1 Protocolo de comunicación serie

Dirección	Formato (binario) y significado
Arduino → MATLAB	`'V'` | `float(4B)` | `float(4B)` | `float(4B)` | `'\n'` *ángulo [°] · velocidad [°/s] · PWM aplicado*
MATLAB → Arduino	`'A'` | `int16(2B)` | `'\n'` *señal de control* $u \in [-255, 255]$

Frecuencia de envío: **100 Hz** ($T_s = 10$ ms). Formato binario (little-endian).

4.2 Pines del Arduino / Shield

Pin Arduino	Señal	Descripción
4	CS (SPI)	Chip Select encoder AS5047P
11	MOSI	SPI MOSI
12	MISO	SPI MISO
13	CLK	SPI Clock
9	EnA (PWM)	Enable / PWM canal A del driver
7	Ain1	Dirección motor (bit 1)
8	Ain2	Dirección motor (bit 2)

4.3 Conversión señal de control → tensión media

El driver TB6612FNG conmuta la tensión de alimentación mediante PWM. La **tensión media** aplicada al motor es proporcional al ciclo de trabajo:

$$\overline{V}_{\text{motor}} = \frac{u}{255} \cdot V_{\text{alim}}$$

donde V_{alim} es la tensión de alimentación del driver (**8,4 V** recomendados con fuente externa; **≈5 V** solo con USB).

Ejemplos con $V_{\text{alim}} = 8{,}4\,\text{V}$: $u = 255 \Rightarrow 8{,}4\,\text{V}$, $u = 128 \Rightarrow 4{,}2\,\text{V}$, $u = 0 \Rightarrow$ motor parado. El signo de u determina la dirección de giro.

5 Resolución de problemas frecuentes

✘ El puerto serie no aparece en MATLAB

- Comprobar que el cable USB está bien conectado.
- En Windows: abrir el Administrador de dispositivos y buscar *"Puertos COM y LPT"*. Si aparece con un triángulo amarillo, reinstalar el driver CH340 o CP2102.
- En Mac: ejecutar `ls /dev/tty.usb*` en Terminal.
- Reiniciar MATLAB después de conectar el Arduino.

✘ El motor no gira

- Verificar que la alimentación está encendida y correctamente conectada.
- Comprobar que la señal de control enviada desde MATLAB no es 0.
- Revisar que los cables del motor están bien conectados al driver.
- Si la señal es muy pequeña ($|u| < 30$) el motor puede no vencer la zona muerta (fricción estática). Es normal; se aborda en la práctica Lab 02.

✘ El encoder no responde / posición siempre 0

- Comprobar el conector JST del encoder (7 pines): debe estar completamente encajado.
- Abrir el Monitor Serie del Arduino IDE a 115200 baudios: si no salen datos, el encoder no se inicializa. El firmware imprime un mensaje de error en ese caso.
- Verificar que el imán del eje está correctamente alineado sobre el encoder (distancia de aire recomendada: 0.5 – 1.5 mm).

✘ Simulink se desconecta a mitad del experimento

- Suele ocurrir si hay otro programa usando el mismo puerto serie (p.ej., el Monitor Serie del Arduino IDE). **Cerrar el Monitor Serie antes de lanzar Simulink.**
- Si el problema persiste, desconectar y reconectar el USB, y relanzar el modelo.

✘ La señal de velocidad es muy ruidosa

- Es normal: la velocidad se obtiene como diferencia de posición dividida por T_s, lo que amplifica el ruido cuantificado del encoder.
- Solución: aplicar un filtro paso bajo de primer orden en MATLAB/Simulink con constante de tiempo $T_f \approx 0.02 - 0.05$ s. Se practica en Lab 01.

6 Repositorio y recursos

Recurso	Ubicación
Repositorio completo	`github.com/DocMuerde/Motor-CC-Control-Lab`
Firmware Arduino	`firmware/arduino/main/main.ino`
Scripts MATLAB	`software/matlab/`
Modelo Simulink	`software/matlab/MotorCC_V3.slx`
Guía de prácticas Lab 01	`labs/lab01/lab01_isa.pdf`
Guía de prácticas Lab 02	`labs/lab02/lab02_isa.pdf`
Planos PCB	`hardware/pcb/`
Piezas 3D	`hardware/3d-printing/`

■ Nota

Para clonar el repositorio completo:

```
git clone https://github.com/DocMuerde/Motor-CC-Control-Lab.git
```